

EndtermRenwed\Group5\server.ino

```
1  /**
2   * @file server.ino
3   * @brief Server-side code for a two-factor authentication (2FA) secure vault system.
4   *
5   * This Arduino sketch acts as the central controller for a secure vault.
6   * It listens for requests from a client via an nRF24L01 radio module,
7   * validates credentials, and monitors a suite of security sensors.
8   *
9   * --- Features ---
10  * 1. **RF Communication:** Receives requests from and sends responses/commands to a
    client.
11  * 2. **Authentication Handling:** Validates passwords and 2FA game coordinates sent by
    the client.
12  * 3. **Intrusion Detection:** Monitors multiple sensors to detect security breaches:
13  * - Laser Tripwire (Laser Diode + LDR)
14  * - Vibration/Shock Sensor (ADXL345 Accelerometer)
15  * 4. **Environmental Monitoring:** Uses a DHT11 sensor for temperature/humidity and a
    fan for ventilation.
16  * 5. **Master Lock Mode:** Enters a lockdown state upon intrusion, sounding an alarm and
    notifying the client.
17  * 6. **Real-Time Clock (RTC):** Keeps track of time for logging and scheduled events.
18  * 7. **Servo Control:** Operates a server-side locking mechanism.
19  *
20  * --- Hardware ---
21  * - Arduino (or compatible board)
22  * - nRF24L01 Radio Module
23  * - Servo Motor
24  * - Buzzer
25  * - DHT11 Temperature & Humidity Sensor
26  * - ADXL345 Accelerometer
27  * - LDR (Light Dependent Resistor)
28  * - Laser Diode
29  * - Fan/Motor for ventilation
30  * - DS3231 Real-Time Clock Module
31  */
32
33  //=====
34  // LIBRARIES
35  //=====
36  #include <SPI.h>          // Required for nRF24L01 communication
37  #include <RF24.h>        // Main library for the nRF24L01 radio module
38  #include <Wire.h>        // Required for I2C communication (RTC, Accelerometer)
39  #include <RTClib.h>      // For interfacing with the DS3231 RTC
40  #include <DHT.h>         // For the DHT temperature and humidity sensor
41  #include <Servo.h>       // For controlling the servo motor
42
43  //=====
44  // PIN & CONSTANT DEFINITIONS
45  //=====
46
47  // --- nRF24L01 Radio Pins ---
48  #define CE_PIN 7
```

```

49 #define CSN_PIN 8
50
51 // --- Component Pins ---
52 #define SERVO_LOCK_PIN 6 // PWM pin for the server-side lock servo
53 #define BUZZER_PIN 4 // Pin to activate the alarm buzzer
54 #define DHTPIN 2 // Data pin for the DHT11 sensor
55 #define MOTORPIN 3 // Pin to control the ventilation fan/motor
56 #define LDR_PIN A0 // Analog pin to read the LDR value for the tripwire
57 #define LASER_TRANSMITTER 5 // Pin to power the laser diode for the tripwire
58
59 // --- RF & Authentication Credentials ---
60 RF24 radio(CE_PIN, CSN_PIN);
61 const byte addresses[][6] = {"00001", "00002"}; // Pipe addresses (must be inverse of
client)
62 const char* authorizedPW = "1234"; // The correct password
63 const int authorizedX = 3; // The correct X-coordinate for the 2FA
game
64 const int authorizedY = 3; // The correct Y-coordinate for the 2FA
game
65 // --- State Variables & Objects ---
66 Servo servoLock;
67 bool isMasterLocked = false; // If true, the system is in a critical lockdown
state
68 bool securitySystemArmed = true; // If true, intrusion sensors are active
69 int ADXL345 = 0x53; // I2C address for the accelerometer
70 float X_out, Y_out, Z_out; // Variables to store accelerometer data
71 RTC_DS3231 rtc; // Create an RTC object
72 DHT dht(DHTPIN, DHT11); // Create a DHT sensor object
73
74 // --- Sensor Thresholds & Timers ---
75 const float TEMP_THRESHOLD = 32.0; // Temp in Celsius to trigger ventilation
76 const float HUMIDITY_THRESHOLD = 62.0; // Humidity % to trigger ventilation
77 const int SENSOR_THRESHOLD = 300; // LDR value below which the laser is considered
"tripped"
78 const float acclThreshold = 1.50; // g-force magnitude on Z-axis to trigger brute-
force alert
79 unsigned long previousMillis = 0; // Used for non-blocking sensor checks
80 const long interval = 5000; // Interval for periodic tasks (5 seconds)
81 bool emailSent = false; // Flag for a one-time scheduled event
82
83 //=====
84 // DATA STRUCTURES (Must match client)
85 //=====
86
87 /**
88  * @enum RequestReason
89  * @brief Defines the purpose of a request received from the client.
90  */
91 enum RequestReason : uint8_t {
92     AUTH_REQUEST = 0, DISARM_REQUEST = 1, ARM_REQUEST = 2, RESET_REQUEST = 3,
93     DOT_MATRIX_AUTH_REQUEST = 4, OPEN_LOCK_REQUEST = 5, CLOSE_LOCK_REQUEST = 6,
94     INITIALIZATION = 7
95 };

```

```

96  /**
97   * @struct RequestData
98   * @brief The data packet structure for communication. Must be identical on the client.
99   */
100 struct RequestData {
101     char password[8];
102     RequestReason reason;
103 };
104
105
106 //=====
107 // SETUP FUNCTION
108 //=====
109
110 /**
111  * @brief Main setup function, runs once at startup.
112  */
113 void setup() {
114     Serial.begin(9600);
115
116     // --- Initialize Hardware ---
117     servoLock.attach(SERVO_LOCK_PIN);
118     servoLock.write(0); // Start with the lock engaged
119
120     pinMode(MOTORPIN, OUTPUT);
121     pinMode(BUZZER_PIN, OUTPUT);
122     pinMode(LASER_TRANSMITTER, OUTPUT);
123     dht.begin();
124
125     // --- RTC Alarm ---
126     rtc.disable32K();
127     rtc.clearAlarm(1);
128     rtc.clearAlarm(2);
129     DateTime alarmTime(0, 0, 0, 12, 12, 0); // Y,M,D are placeholders
130     if (!rtc.setAlarm1(alarmTime, DS3231_A1_Hour)) {
131         Serial.println("Error, alarm not set!");
132     } else {
133         Serial.println("Alarm is set for 12:12:00. Waiting...");
134     }
135
136     // --- Initialize I2C Devices ---
137     Wire.begin();
138     if (!rtc.begin()) { // Initialize Real-Time Clock
139         Serial.println("FATAL: Couldn't find RTC");
140         abort(); // Halt execution if RTC is not found
141     }
142     // Initialize ADXL345 Accelerometer
143     Wire.beginTransmission(ADXL345);
144     Wire.write(0x2D); // Access power control register
145     Wire.write(8);    // Set to measurement mode
146     Wire.endTransmission();
147
148     // --- Initialize Radio Communication ---
149     radio.begin();

```

```

150  radio.setChannel(108);
151  radio.openReadingPipe(1, addresses[0]); // Use address 0 for reading from client
152  radio.openWritingPipe(addresses[1]);    // Use address 1 for writing to client
153  radio.setPALevel(RF24_PA_MIN);
154  radio.startListening(); // Start in listening mode
155
156  Serial.println("Server online. Security is ARMED.");
157 }
158
159 //=====
160 // MAIN LOOP
161 //=====
162
163 void loop() {
164     // Continuously check for intrusions if the system is armed
165     handleIntrusion();
166
167     // Turn the laser on or off based on the security system's armed state
168     digitalWrite(LASER_TRANSMITTER, securitySystemArmed);
169
170     // If system is master-locked, sound the alarm and halt all other operations.
171     // The only way out is a RESET command from the client.
172     if (isMasterLocked) {
173         digitalWrite(BUZZER_PIN, HIGH);
174     }
175
176     // --- Task 1: Process incoming commands from the client ---
177     if (radio.available()) {
178         RequestData req;
179         radio.read(&req, sizeof(RequestData)); // Read the incoming data packet
180
181         // Use a switch to handle the request based on its reason
182         switch (req.reason) {
183             case RESET_REQUEST:
184                 Serial.println("RESET command received! Unlocking system and re-arming.");
185                 isMasterLocked = false;          // Exit master lock mode
186                 securitySystemArmed = true;       // Re-arm the system
187                 digitalWrite(BUZZER_PIN, LOW);    // Turn off the alarm
188                 break;
189
190             case AUTH_REQUEST: // First factor: Password check
191                 if (!isMasterLocked) {
192                     Serial.print("RF_RX: AUTH request with password: ");
193                     Serial.println(req.password);
194                     radio.stopListening();
195                     // Compare received password with the authorized one
196                     if (strcmp(req.password, authorizedPW) == 0) {
197                         radio.write("VALID", sizeof("VALID"));
198                     } else {
199                         radio.write("INVALID", sizeof("INVALID"));
200                     }
201                     radio.startListening();
202                 }
203                 break;

```

```

204
205     case DOT_MATRIX_AUTH_REQUEST: // Second factor: Game coordinates check
206         if (!isMasterLocked) {
207             Serial.print("RF_RX: Dot matrix coords: ");
208             Serial.println(req.password);
209             int parsedX, parsedY;
210             sscanf(req.password, "%d|%d", &parsedX, &parsedY); // Parse "X|Y" string
211             radio.stopListening();
212             // Compare coordinates with authorized ones
213             if (parsedX == authorizedX && parsedY == authorizedY) {
214                 radio.write("VALID", sizeof("VALID"));
215             } else {
216                 radio.write("INVALID", sizeof("INVALID"));
217             }
218             radio.startListening();
219         }
220         break;
221
222     case OPEN_LOCK_REQUEST:
223         if (!isMasterLocked) {
224             Serial.println("Info: Received command to UNLOCK server-side servo.");
225             servoLock.write(180); // Open the lock
226         }
227         break;
228
229     case CLOSE_LOCK_REQUEST:
230         Serial.println("Info: Received command to LOCK server-side servo.");
231         servoLock.write(0); // Close the lock
232         break;
233
234     case DISARM_REQUEST:
235         securitySystemArmed = false;
236         Serial.println("Info: Security system DISARMED by client.");
237         break;
238
239     case ARM_REQUEST:
240         securitySystemArmed = true;
241         Serial.println("Info: Security system ARMED by client.");
242         break;
243
244     case INITIALIZATION: // Received when client comes online
245         Serial.println("Info: Received client initialization. Resetting to secure
state.");
246         servoLock.write(0);
247         securitySystemArmed = true;
248         break;
249     }
250 }
251
252 // --- Task 2: Perform periodic checks (non-blocking) ---
253 unsigned long currentMillis = millis();
254 if (currentMillis - previousMillis >= interval) {
255     previousMillis = currentMillis;
256     DateTime now = rtc.now(); // Get current time from RTC

```

```

257
258 // Sub-task A: Check temperature/humidity and control ventilation
259 float t = dht.readTemperature();
260 float humidity = dht.readHumidity();
261 if (!isnan(t)) { // Check for a valid sensor reading
262     Serial.print("Info: Temp is ");
263     Serial.print(t);
264     Serial.print(" C | Humidity is ");
265     Serial.println(humidity);
266     // Turn on fan if thresholds are exceeded
267     if (t > TEMP_THRESHOLD || humidity > HUMIDITY_THRESHOLD) {
268         Serial.println("Temperature/humidity is high! Ventilation functioning!!");
269         digitalWrite(MOTORPIN, HIGH);
270     } else {
271         digitalWrite(MOTORPIN, LOW);
272     }
273 }
274
275 // Sub-task B: Report current time to Serial Monitor for logging
276 Serial.print("Info: Time is ");
277 Serial.print(now.hour()); Serial.print(':');
278 if (now.minute() < 10) { Serial.print('0'); }
279 Serial.print(now.minute()); Serial.print(':');
280 if (now.second() < 10) { Serial.print('0'); }
281 Serial.println(now.second());
282 Serial.println("---");
283 }
284
285 // --- Task 3: Check for scheduled events ---
286 // This is a placeholder for a scheduled action, like sending an email via a connected
ESP.
287 if (!emailSent && rtc.alarmFired(1)) {
288     rtc.clearAlarm(1);
289     Serial.println("SCHEDULED_EVENT: SEND_EMAIL"); // Command for a listening Python
script
290     emailSent = true; // Ensure this only runs once
291 }
292 }
293
294 //=====
295 // HELPER FUNCTIONS
296 //=====
297
298 /**
299  * @brief Checks intrusion sensors and triggers a master lockdown if breached.
300  */
301 void handleIntrusion() {
302     if (securitySystemArmed && !isMasterLocked) {
303         // Check 1: Laser tripwire. Low LDR value means the beam is broken.
304         bool isLaserTripped = analogRead(LDR_PIN) < SENSOR_THRESHOLD;
305
306         // Check 2: Brute force. A sudden spike in Z-axis acceleration indicates a shock.
307         bool isBruteForced = fabs(getZAccelerationMagnitude()) > acc1Threshold;
308

```

```

309     if (isLaserTripped || isBruteForced) {
310         isMasterLocked = true; // CRITICAL: Engage master lock
311         Serial.println("CRITICAL: INTRUSION DETECTED! SYSTEM LOCKED!");
312
313         // Notify the client immediately to force its own lockdown
314         radio.stopListening();
315         radio.write("LOCKDOWN", sizeof("LOCKDOWN"));
316         radio.startListening();
317     }
318 }
319 }
320
321 /**
322  * @brief Reads the raw X, Y, and Z acceleration values from the ADXL345.
323  */
324 void getAccelerations() {
325     Wire.beginTransaction(ADXL345);
326     Wire.write(0x32); // Start reading from the data register
327     Wire.endTransmission(false);
328     Wire.requestFrom(ADXL345, 6, true); // Request 6 bytes of data
329     // Read and combine bytes for each axis, then scale the value
330     X_out = (Wire.read() | Wire.read() << 8) / 256.0;
331     Y_out = (Wire.read() | Wire.read() << 8) / 256.0;
332     Z_out = (Wire.read() | Wire.read() << 8) / 256.0;
333 }
334
335 /**
336  * @brief A convenience function to get the absolute magnitude of Z-axis acceleration.
337  * @return The absolute value of the Z-axis acceleration in g-forces.
338  */
339 float getZAccelerationMagnitude() {
340     getAccelerations();
341     return fabs(Z_out);
342 }

```